

Sizing, borders, and rounded corners

In the last lesson, you learned that every element is a box. You added padding inside boxes, borders around boxes, and margin outside boxes. Now you can decide how big a box should be.

Sizing properties help answer practical questions:

- Should this card be a specific width?
- Should it be allowed to shrink on smaller screens?
- Should it have a minimum height?
- Should the box feel sharp, rounded, flat, or lifted?

This lesson turns the box model into practical card styling.

Start with a simple card

Use a small plan card as the practice surface. Put this markup in `index.html` :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Sizing practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <article class="plan-card">
        <p class="eyebrow">Starter plan</p>
        <h1>Build better habits</h1>
        <p class="summary">A simple plan for learning CSS through small, visible changes</p>
        <p class="price">$12 per month</p>
        <a class="card-action" href="/start">Start learning</a>
      </article>
    </main>
  </body>
</html>
```

Then give it a basic stylesheet:

css

```
body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}
```



```
.page {
  max-width: 720px;
  margin: 40px auto;
}

.plan-card {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
}

.eyebrow {
  color: #2563eb;
  font-weight: 700;
}

.summary {
  color: #4b5563;
  line-height: 1.6;
}

.price {
  font-size: 24px;
  font-weight: 700;
}

.card-action {
  color: #2563eb;
  text-decoration: none;
  font-weight: 700;
}
```

available space. That is not wrong because block elements often take the available width, but many UI pieces, like cards and forms, need a more deliberate size.

Set a width

The `width` property sets how wide an element should be.

Set `width: 320px` on the card:

css

```
.plan-card {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
  width: 320px;
}
```

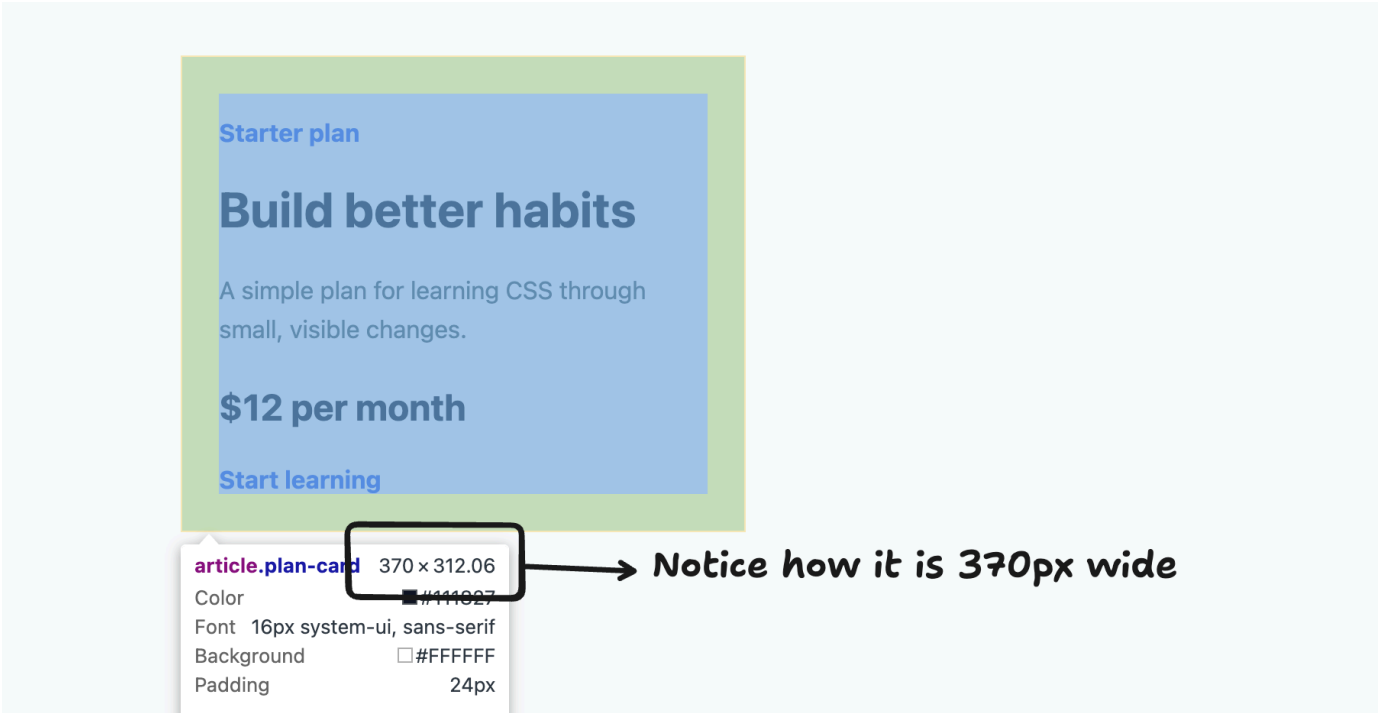
Reload the page. The card should now look narrower, but there is a box model surprise here.

By default, `width: 320px` sets the width of the **content area**. Padding and border are added on top of that. So the visible card can be wider than 320px.

ⓘ Width uses the content box by default - Width is set on the content area by default which means that even though we define the width to

be **320px**, the actual width of the element will be more than that because padding and borders are added on top of that width.

Notice how the actual width in browser is **370px** instead of 320px.

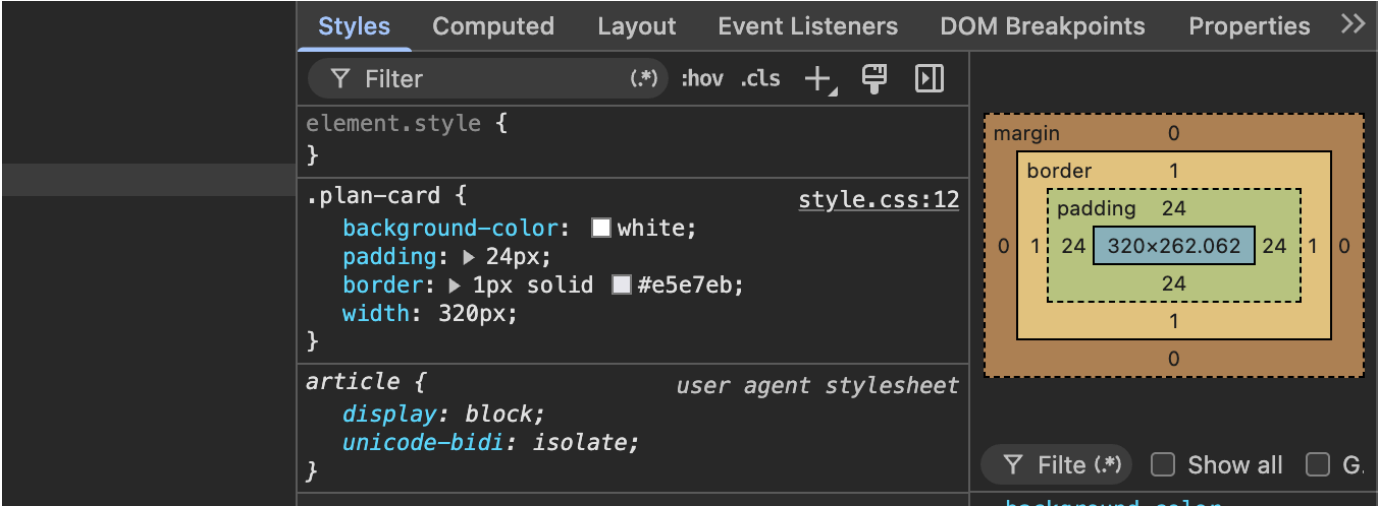


If the card has **24px** of padding on both sides and a **1px** border on both sides, the visible width becomes:

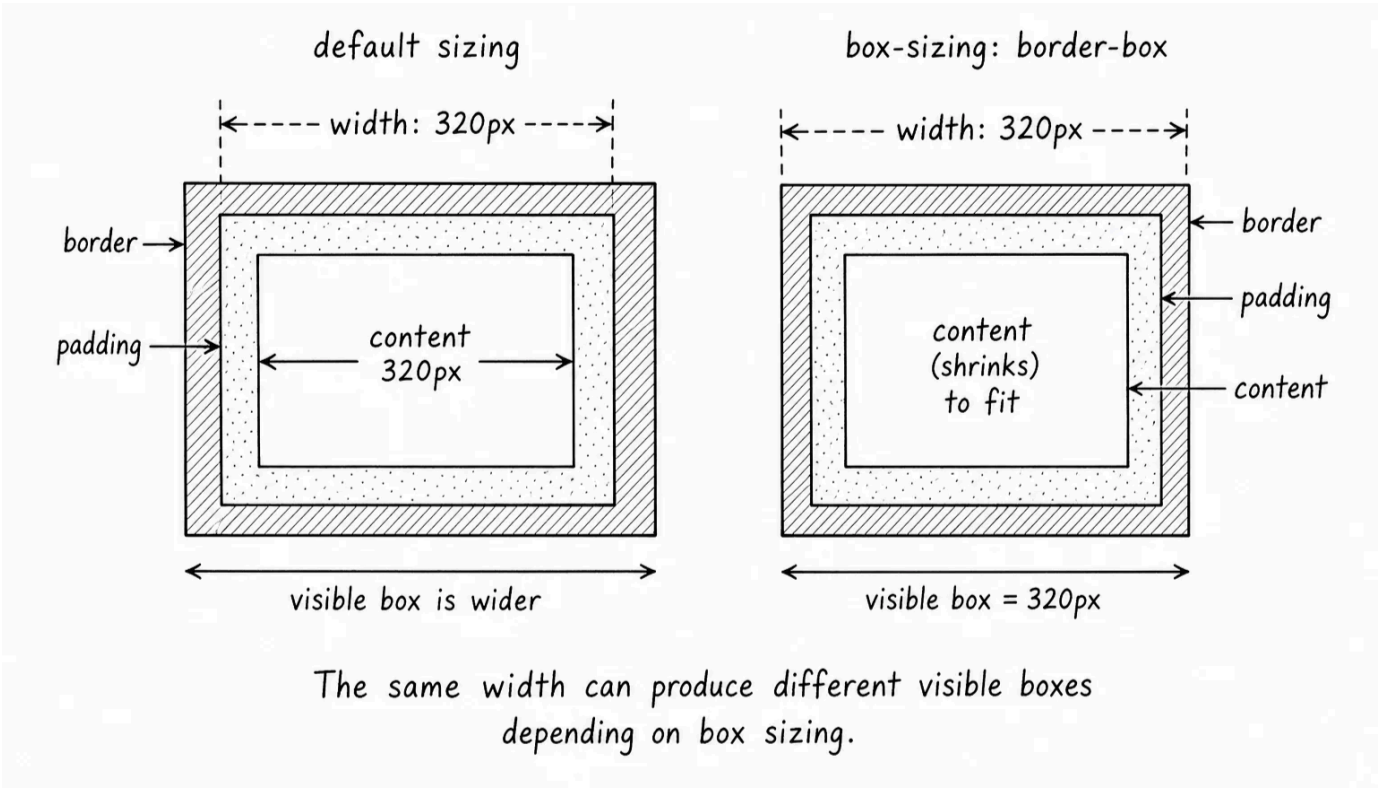
```
text

320px content
+ 24px left padding
+ 24px right padding
+ 1px left border
+ 1px right border
= 370px visible box
```

Inspect the card in DevTools if you want to confirm the numbers. The box model diagram should show the content width, padding, and border separately.



This is where **box-sizing** matters because it controls how the browser applies width. If you want the set width to include the padding and border, use **box-sizing: border-box** .



Use border-box sizing

Most modern projects make sizing easier with this rule:

```
CSS

* {
  box-sizing: border-box;
}
```

The `*` selector matches every element. `box-sizing: border-box` tells the browser to include content, padding, and border inside the set width.

The default behavior has a name too: `box-sizing: content-box`. With `content-box`, width applies to the content only, and padding and border are added outside it. `border-box` flips that: padding and border are pulled inside the width.

Add it near the top of your CSS:

```
CSS

* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}
```

Reload the page and inspect `.plan-card` in DevTools. With `border-box`, the visible card width is easier to reason about: the content, padding, and border fit inside the `320px` width.

💡 Use border-box in real projects - `box-sizing: border-box` is a practical default because it makes width and height include padding

and border, which usually matches how people expect boxes to behave.

Prefer max-width for flexible boxes

A fixed `width` can be useful for a small demo, but it can be too rigid in real pages. A card that is exactly `320px` wide may be fine on one screen and awkward on another. A width of `800px`, for example, might look fine on a desktop but break on a smaller screen.

Often, you want the box to be flexible up to a limit. This is where a combination of `width` and `max-width` comes in.

You can also use percentages for `width`: `100%` means the full available width, `80%` means 80% of the available width, and `50%` means half.

If you want the card to work on smaller and larger screens, set `width: 100%` so it can use the available space, then constrain it with `max-width: 420px`:

CSS

```
.plan-card {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
  width: 100%;
  max-width: 420px;
}
```

`width: 100%` lets the card use the available space, and `max-width: 420px` stops it from becoming wider than `420px`. On narrow screens, the card can shrink with the page. On wider screens, it stops at `420px`.

For a block element like `<article>`, `width: 100%` is often technically redundant because block elements already use the available width. You will still see this pattern in real projects because it makes the intent explicit and also works well when the element is not a normal block.

This is a common pattern for cards, forms, articles, and centered content.

You may also see `min-width`, which sets a minimum width. Use it carefully, because a minimum width can make layouts harder on small screens.

Avoid fixed height for text

The `height` property sets an exact height.

That can be useful for things like icons, avatars, thumbnails, and fixed-size controls. But fixed heights can cause trouble for text-heavy boxes.

Avoid this for normal cards:

CSS

```
.plan-card {  
  height: 180px;  
}
```

If the text gets longer, it may overflow or feel squeezed.

Use `min-height` when you want a box to have at least a certain height while still allowing it to grow:

CSS

```
.plan-card {  
  min-height: 260px;  
}
```

`min-height` says, "be at least this tall, but grow if the content needs more room."

You may also see `max-height`, which sets an upper limit. Once content is taller than that limit, overflow becomes part of the problem, which you will learn later.

 **Avoid fixed heights for normal text** - If a box contains text, be careful with `height` because text can wrap, translations can be longer, and users may increase font size. `min-height` is usually safer.

Round the corners

The `border-radius` property rounds the corners of a box.

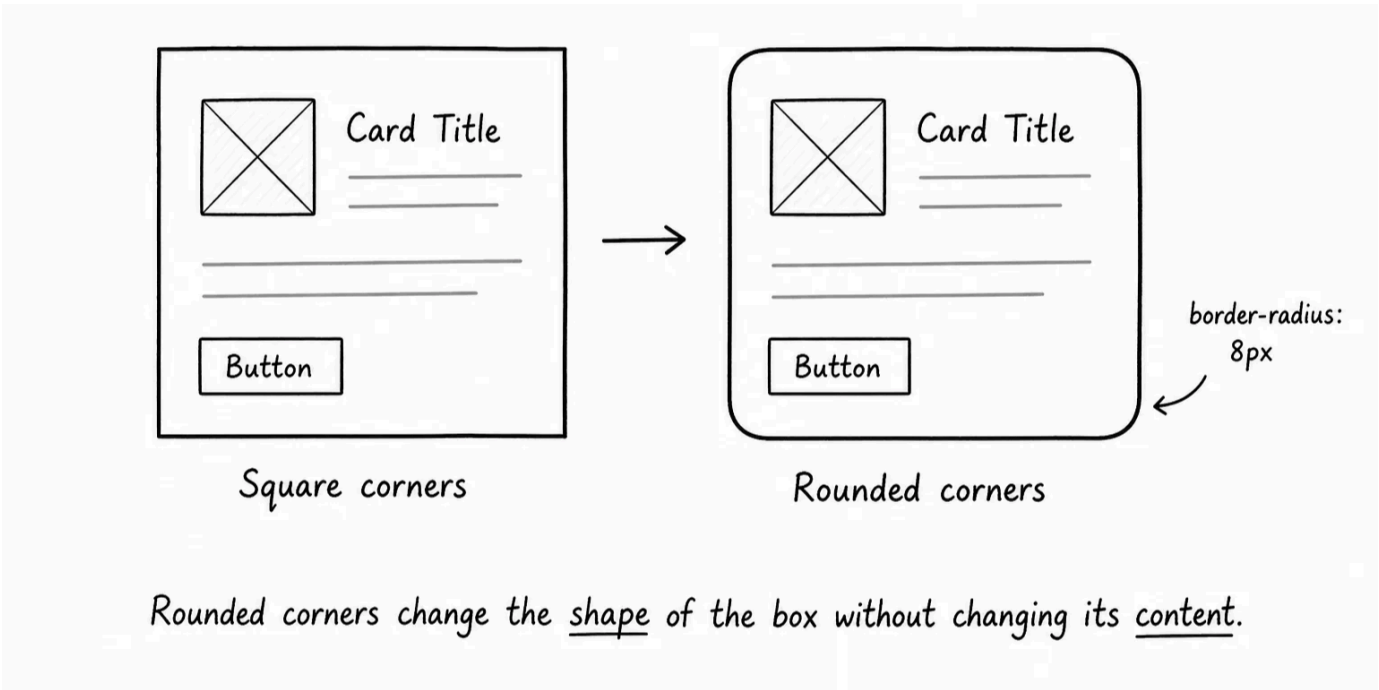
Add the radius to the card:

CSS

```
.plan-card {  
  background-color: white;  
  padding: 24px;  
  border: 1px solid #e5e7eb;  
  width: 100%;  
  max-width: 420px;  
  min-height: 260px;  
  border-radius: 8px;  
}
```

Small radius values make a box feel slightly softer. Large values can create pills or circles, depending on the element shape.

For beginner UI work, values like `4px`, `6px`, `8px`, or `12px` are usually enough.



Add a light shadow

The `box-shadow` property draws a shadow around a box.

Add a subtle shadow to the card:

CSS

```
.plan-card {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
  width: 100%;
  max-width: 420px;
  min-height: 260px;
  border-radius: 8px;
  box-shadow: 0 8px 24px rgba(15, 23, 42, 0.08);
}
```

The shadow value has four parts:

- `0` : horizontal offset
- `8px` : vertical offset
- `24px` : blur
- `rgba(15, 23, 42, 0.08)` : shadow color

Positive vertical offset pushes the shadow down. That is why cards usually have shadow below them: it suggests the card is lifted above the page.

Shadows usually use `rgba()` so the alpha value keeps them subtle. A solid `rgb()` or hex shadow can feel too heavy.

You do not need to memorize shadow math. The practical habit is to keep shadows subtle. A heavy shadow can make a simple UI feel noisy.

Borders, radius, and shadows are visual styling. They should support the content, not compete with it.

✦ AI Tutor

Explain how subtle and heavy box shadows change the feeling of a card. Use the card from this lesson as the example, and focus on when a shadow supports the content versus when it becomes distracting.

Try it

The card now has sizing, border, radius, and shadow. Try the following:

- Change `.plan-card` from `width: 100%; max-width: 420px;` back to `width: 320px;`. Inspect the visible width with and without `box-sizing: border-box`.
- Change `max-width` from `420px` to `520px`.
- Change `min-height` from `260px` to `160px`, then add more text to the summary.
- Change the border color from `#e5e7eb` to `#9ca3af`, then change it back.
- Change `border-radius` from `8px` to `0`, then to `16px`.
- Remove `box-shadow`, then add it back.
- Make the shadow stronger by changing `0.08` to `0.2`, then decide which version feels cleaner.

For an extra challenge, add a second card under the first one:

html

```
<article class="plan-card secondary-card">
  <p class="eyebrow">Team plan</p>
  <h1>Practice together</h1>
  <p class="summary">A shared plan for reviewing lessons, comparing solutions, and build
  <p class="price">$29 per month</p>
  <a class="card-action" href="/team">Start as a team</a>
</article>
```

Then write a `.secondary-card` rule that changes only the border color or shadow.

This uses the multi-class pattern from Lesson 2. The card gets all the `.plan-card` styles and adds your `.secondary-card` variant on top.

The goal is to connect sizing and visual polish to the box model. Width sets the box size, `box-sizing` changes how that size is calculated, and radius or shadow changes how the box feels.

Test Your Understanding

What does width do?

[Click to Reveal the Answer](#)

✔ Already Know that

✖ Didn't Know that

⏩ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →